

Essenza del paradigma funzionale $\rightarrow$ 5 programmi sono formati da funzioni ed espressioni.

Non vi sono assegnamenti e i valori sono immutabili e legati ad un identificatore. Senza assegnamenti non possiamo avere dei cicli, di conseguenza:

- C'è la ricorsione;
- modifichiamo l'ambiente, anziché la memoria;
- creiamo id con stesso nome mediante chiamate ricorsive / mascherando le precedenti

Sistema di tipi $\rightarrow$ ML è fortemente e staticamente tipato (**type checking a compile time**).

- Inoltre non necessita la specificazione dei tipi, capita da solo (**type inference**).
- Sfrutta sia la **structural equivalence** che la **name equivalence**;
- Permette di definire liste, alberi...;
- Supporta il polimorfismo parametrico
- " l'encapsulation
- Tiene la gerarchia dei tipi (e l'**ereditarietà**)

Implementazione di PL:

- Può essere interpretato con **Standard PL** (usiamo questo);
- Può essere compilato

Tipi primitivi

■ int: gli interi

0, 1, ~1, 2, ~2, ... 0xff, ~0x32, ...

Notare che si usa ~ invece del segno meno (-). Alcuni operatori:

+, -, *, **div** mod, =, <, ...

■ word: unsigned integers, prefisso 0w *Parole macchina*

0w44, 0w15, ... 0wxff, ...

■ real *float*

3.14, 2.0, 0.1E6, ...

Alcuni operatori su real:

+, -, *, **<**, ...

■ string

"abc", "123", ...

Alcuni operatori su string:

size, =, <, ...

■ char

#"a", #"n", #"163", ...

Alcuni operatori su char:

ord, **chr**, =, <, ...

■ bool *ASCII $\leftrightarrow$ char*

true, false

Alcuni operatori su bool:

not, **andalso**, **orelse**, =

AND e OR logics

Dichiarazione e scoping in ML (Funzioni) $\rightarrow$ ML ha diversi modi per definire e chiamare funzioni

Modo tradizionale

- $\text{fun quad2 } x = x * x;$ $\rightarrow$ corpo dell'espressione e return

$\text{val quad2} = \text{fn} : \text{int} \rightarrow \text{int}$ associa il valore restituito da $\text{fn} : \text{int} \rightarrow \text{int}$ a quad2

- $\text{quad2}(3);$ // Le parentesi sono opzionali
 $\text{val it} = 9 : \text{int}$

Fun dichiara la funzione e aggiunge all'ambiente il id quad2 , associato alla funzione $\text{int} \rightarrow \text{int}$

chiamando solo $\text{quad2} \dots$

- $\text{quad2};$
 $\text{val it} = \text{fn} : (\text{int} \rightarrow \text{int})$ vediamo l'associazione

Annotazione esplicita dei tipi (serve, anticipa la type inference)

- $\text{fun quad2 } (x : \text{int}) : \text{int} = x * x;$

val quadr = fn : int → int

Recursione

- fun fatt x = if x = 0 then 1 else x * fatt (x-1);

val fatt = fn : int → int

Il costrutto if-then-else è un'espressione con un valore
return, se x=0 ho 1, altrimenti...

Altre dichiarazioni → Val aggiunge un id all'ambiente con un valore

- val x = 2 + 2;

val x = 6 : int;

- x + 2;

val it = 6 : int

Grammatica delle dichiarazioni viste sinora

```
<declaration> ::=  
  val <id name> = <expression> |  
  fun <func name> <argument>* = <expression>
```

static

Scoping → L'equivalente dei blocchi in ML è let in end

let

$\langle \text{dichiarazioni} \rangle$
 in
 $\langle \text{Espressioni} \rangle$
 end

hanno valore solo qui

Esempio

```

- let val x=2 in 3*x end;
val it = 6 : int

- x;
Error: unbound variable or constructor: x

- let val x=2 in
  let val x=3 in (* questa def. maschera la precedente *)
    3*x
  end
end;
val it = 9 : int
  
```

Ambiente non locale delle funzioni

- val x=0;
 Val x=0 : int

- let val x=1 in
 let fun f(y) = x * y in
 f(0)
 end
 end
 Val it = 1 : int

x non locale alla funzione

Clausola local → È composta da local in end, è simile al let

```
local
  < dichiarazioni >
in
  < altre dichiarazioni >
end
```

Esempio

```
let
  local
    val x = 42
  in
    val y = x div 2
  end
in
  y + 1
end
```

Tipi strutturati:

- È possibile definire n-pla mettendo i valori tra parentesi
- Il prodotto cartesiano si indica con *
- Con # estraiamo l'i-esimo elem. della n-pla (si conta da 1)

```
- (1+1, "A");] def di una n-pla
val it = (2, "A") : int * string] prodotto cartesiano

- val x = (1, "A", 3.5);
val x = (1, "A", 3.5) : int * string * real

- (#1(x)); Non si conta da 0
val it = 1 : int

- #2(x);
val it = "A" : string

- #3(x);
val it = 3.5 : real
```

Record → Insieme di espressioni $\langle \text{nome} \rangle = \langle \text{valore} \rangle$

- Val $R = \{ \text{nome} = \text{"Mario"}, \text{nato} = 1998 \};$
Val $R = \{ \text{nato} = 1998, \text{nome} = \text{"Mario"} \} : \{ \text{nato} : \text{int}, \text{nome} : \text{string} \}$

Il tipo viene rappresentato così

Il valore associato al nome N si estrae con $\#N$

```
- #nome(r);  
val it = "Mario" : string  
  
- #nato(r);  
val it = 1998 : int
```

L'ordine delle coppie non è rilevante

```
- {nome="Mario", nato=1998} = {nato=1998, nome="Mario"};  
val it = true : bool
```

Siciliazione di tipo → Definizioni di tipi nuovi come la `typedef`, con la clausola **type**

- Type `coord = real * real`
Type `coord = real * real`

Il compilatore è un po' ostinato e a volte non capisce il tipo definito

```
- val x = (3.0, 4.0); (* senza aiutino *)  
val x = (3.0, 4.0) : real * real  
- val x:coord = (3.0, 4.0); (* con aiutino *)  
val x = (3.0, 4.0) : coord
```

Un coord è risolvere tutto

coord e real * real sono equivalenti (sono reciprocamente compatibili)
↳ è compatibile con altri nuovi tipi real * real

Datatypes e costruttori → con datatypes definiamo costruttori di data objects

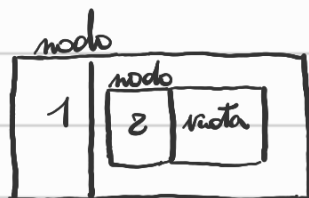
```
- datatype color = red | green | blue;  
datatype color = blue | green | red  
  
- val c = red;  
val c = red : color
```

| sarebbe l'OR

red, green e blue sono costruttori e possibili valori del tipo color (enum del c)

Costruttori con argomenti → Definiamo una lista di interi con la def. ricorsiva

- 1) È vuota;
- 2) È un nodo con un intero e una lista di interi



Occorrono due costruttori

- datatype listaint = vuoto | nodo of int * listaint ;
datatype listaint = nodo of int * listaint | vuoto

- val L = nodo (1, nodo (2, vuoto));
val L = nodo (1, nodo (2, vuoto)) : listaint

Proviamo a definire un albero binario con nodi interi

Def. ricorsiva $\Rightarrow$ un albero è :

- 1) vuoto ;
- 2) un albero con un nodo intero e due alberi dello stesso tipo

datatype albero = vuoto | nodoAlb of int * albero * albero

Albero con radice 1, figlio sx 2 (foglia) e figlio destro mancante :

nodoAlb (1, nodoAlb (2, vuoto, vuoto), vuoto)

Tipi mutuamente ricorsivi $\Rightarrow$ cerchiamo di definire un tipo di albero binario i cui nodi con profondità dispari sono interi e i pari sono stringhe

(Supponiamo che la radice abbia profondità 1)

1° Tentativo

datatype albero = vuoto

| nodop of string * albero * albero

| nodod of int * albero * albero

Un nodo dispari può avere figli dispari

Da approfondire

2° Tentativo

datatype nodopari = nodop of string * nododispari * nododispari | vuoto;

↳ dove è definito?

datatype nododispari = nodod of int * nodopari * nodopari | vuoto;

type albero = nododispari;

Ultimo tentativo → Clausola and

datatype nodopari =

nodop of string * nododispari * nododispari | vuoto

and nododispari =

nodod of int * nodopari * nodopari | vuoto;

type albero = nododispari

↳ Alias

// siccome la radice è dispari, devo partire con nodo dispari

and permette di definire e combinare funzioni mutuamente ricorsive

Patterns e matching, utilizzo dei costruttori con argomenti

Per rendere una lista in memoria verificare che sia vuota

```
- val L = nodo(1, nodo(2, vuota));  
val L = nodo (1,nodo (2,vuota)) : listaInt  
  
- L = vuota;  
val it = false : bool
```

In questo caso, c'è qualcosa e magari potrebbe servire il primo elemento.
Con il **pattern matching** possiamo estrarlo.

→ Pattern

```
- val nodo(p,_) = L;      (* assegna a p il 1° elemento di L *)  
val p = 1 : int          (* "_" è una wildcard *)
```

Significato = vedere se L corrisponde al pattern `nodo`, `p` è qualsiasi altra cosa. **"_"** significa qualsiasi cosa

Per ottenere il resto della lista

```
- val nodo(_,r) = L;      (* assegna a r il resto *)  
val r = nodo (2,vuota) : listaInt
```

Funzione che conta gli elementi della lista

```

- fun conta x =
  if x = vuota then 0
  else
    let val nodo(_, r) = x in
      conta(r) + 1
    end;
val conta = fn : listaInt -> int

- conta L;
val it = 2 : int

```

Il compilatore controlla la compatibilità di tipi tra i seguenti :

1) π , 2° argomento del nodo, di tipo listaInt $\Rightarrow$ conta lo accetta e restituisce int

2) Che restituisce int, tutto è corretto

I pattern possono coinvolgere datatype, tuple, record

```

- val a = { nome="Mario", nascita=2002 };
val a = {nascita=2002,nome="Mario"} : {nascita:int, nome:string}
- val { nome=_, nascita=anno } = a;
val anno = 2002 : int

```

Un pattern può contenere più variabili

```

- val nodo(p, r) = L;
val p = 1 : int
val r = nodo (2, vuota) : listaInt

```

Cioè dichiara due identificatori (p e r) in un colpo solo

Un pattern non può contenere la stessa variabile due volte

```

- val (x,x) = (2,2);
stdIn:1.2-1.19 Error:  duplicate variable in pattern(s):  x

```

Si può definire una funzione per casi

```

- fun conta(vuota) = 0
  | conta(nodo(_, r)) = conta(r) + 1;

```

mettendo direttamente i pattern al posto dei parametri formali

L'ordine dei casi è rilevante

Notare l'eleganza e la concisione

Scrivere una funzione ricorsiva che confronta due alberi binari con valori interi e restituisce `true` se sono identici

```
fun alberi_uguali empty empty = true
| alberi_uguali (bnode(v1,l1,r1)) (bnode(v2,l2,r2)) =
  v1=v2 andalso
  alberi_uguali l1 l2 andalso
  alberi_uguali r1 r2
| alberi_uguali _ _ = false;
```

quasi: altro caso

Errori comuni:

```
fun alberi_uguali empty empty = true
| alberi_uguali (bnode(v,l1,r1)) (bnode(v,l2,r2)) =
  alberi_uguali l1 l2 andalso alberi_uguali2 r1 r2
| ...
```

Error: duplicate variable in pattern(s): v

```
fun alberi_uguali empty empty = true
| alberi_uguali (bnode(v1,l1,r1)) (bnode(v2,l2,r2)) =
  v1=v2 andalso
  alberi_uguali l1 l2 andalso
  alberi_uguali2 r1 r2;
```

Warning: match nonexhaustive
 (empty,empty) => ...
 (bnode (v1,l1,r1),bnode (v2,l2,r2)) => ...

101/257

Costrutto case-of $\rightarrow$ simile allo switch case

- case `l` of `valuta` $\Rightarrow$ `true`

| `modo` (`_`, `_`) $\Rightarrow$ `False`;

val `it` = `false` : bool

- L'ordine è rilevante

- Il costrutto case-of crea un'espressione

Scrivere in ML una funzione che, dato un albero binario A come quello visto in precedenza, e un intero N, restituisce l'etichetta del nodo che si raggiunge in N passi visitando l'albero in preordine

datatype btree = empty | bnode of int btree * btree;

fun size empty = 0

| size bnode (_, L, R) = 1 + (size L) + (size R);

fun height empty = 0

| height bnode (_, L, R) = 1 + Int.max (height L, height R)

↓
Funzione di ML, restituisce il massimo tra due interi

Liste built-in $\Rightarrow$ nella programmazione funzionale sostituiscono i vettori

Sono presenti i seguenti costruttori $\Rightarrow$ `nil` (`root`) e `::` (`costruzione effettiva`)

`nil`

`1 :: 2 :: 3 :: nil`

lista con 1, 2, 3



formato equivalente



`[]`

`[1, 2, 3]`

(* sono veramente equivalenti *)

- `[] = nil;`

val it = true : bool

- `1::2::3::nil = [1,2,3];`

val it = true : bool

Funzioni di supporto

- `length` $\Rightarrow$ restituire la lunghezza
- `null` $\Rightarrow$ " true se la lista è vuota
- `hd` e `tl` restituiscono il valore della testa e del resto, rispettivamente
(Se la lista ha un solo valore `hd` restituisce il valore e `tl` restituisce `null`)
- `@` concatena due liste

```
- val L = [1,2,3];
val L = [1,2,3] : int list

- hd L;
val it = 1 : int

- tl L;
val it = [2,3] : int list

- val M = [1,2] @ [3,4];
val M = [1,2,3,4] : int list
```

Funzioni della struttura List

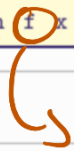
- `List.nth (L,i)` $\Rightarrow$ restituisce l'i-esimo elem (Partecola da L)
- `List.last (L)` $\Rightarrow$ " l'ultimo

Currying, funzioni con più argomenti, bastano?

In realtà in ML ogni funzione ha un solo argomento

```
fun f (x,y) = ...      (* l'argomento è una (singola) coppia *)
```

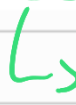
```
fun f x y = ...      (* l'argomento è x ! *)
```

 f restituisce una funzione che prende y e calcola l'espressione dopo =

Esempio semplice

```
- fun f (x,y) = x+y;  
val f = fn : int * int -> int  
  
- fun f' x y = x+y;  
val f' = fn : int -> int -> int
```

Il tipo `int -> int -> int` va inteso come `int -> (int -> int)`

 prende un intero come 1° argomento e restituisce un'altra funzione che a sua volta prende un intero come 2° argomento e restituisce un intero

Visto che ML non prevede la possibilità di passare più argomenti alla volta, l'idea del *currying* è quella di propagare gli argomenti per diverse funzioni, fino ad arrivare all'ultima

```
- fun f (x,y) = x+y;  
val f = fn : int * int -> int  
  
- fun f' x y = x+y;  
val f' = fn : int -> int -> int  
  
- f (3,2);  
val it = 5 : int  
  
- f' 3 2;          (* viene interpretato come (f'3)(2) *)  
val it = 5 : int  
  
- f 3 2;  
Error: operator and operand don't agree  
  
- f' (3,2);  
Error: operator and operand don't agree
```

 non possono essere invertite

Funzioni di ordine superiore -> parte delle funzioni ricorsive che operano su liste, alberi e simili hanno la stessa struttura (modi a parte)

Basta scrivere la funzione che scanda la struttura e fine.

Higher, le funzioni che hanno altre funzioni come parametri sono dette di **ordine superiore** e fanno da loop

Esegue due volte g su un valore x

```
- fun f g x = g(g(x));  
val f = fn : ('a -> 'a) -> 'a -> 'a
```

Tre tipi comuni

- filter;
- map (trasformare);
- aggregare (reduce / fold)

Filter $\rightarrow$ prende una funzione booleana f e una lista L , seleziona gli elementi di L per cui è vero f

```
fun filter f [] = []  
  | filter f (x::y) = if f(x) then x::(filter f y)  
                      else filter f y  
  
(* esempio: seleziona gli elementi negativi da una lista *)  
- let fun neg x = x < 0  
    in filter neg [0, ~1, 3, ~2] end;  
val it = [~1, ~2] : int list  
  
(* esempio: seleziona gli elementi positivi da una lista *)  
- let fun pos x = x > 0  
    in filter pos [0, ~1, 3, ~2] end;  
val it = [3] : int list
```

Map → prende f e una lista L , applica f a tutti gli elementi della lista, ottenendo una nuova lista

```
fun map f [] = []
  | map f (x::y) = (f x)::(map f y)

(* esempio: conversione in lista di reali *)
- map real [1,2,3];
val it = [1.0,2.0,3.0] : real list

(* esempio: conversione in lista di stringhe *)
- map Int.toString [1,2,3];
val it = ["1","2","3"] : string list
```

Reduce / Fold → reduce serve per calcolare, **min**, **max**, **somma**, **prodotto**, **media**

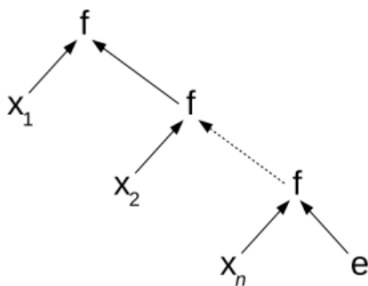
prende una funzione a 2 argomenti f , un **valore finale** e una lista L ed effettua il seguente calcolo

e, anche se viene chiamato
neutro / accumulatore



quindi ad ogni funzione calcolata
"e" avrà sempre il valore
restituito

$\text{reduce } f \text{ e } [x_1, x_2, \dots, x_n] = f(x_1, f(x_2, \dots f(x_n, e) \dots))$



```
fun reduce f e [] = e
  | reduce f e (x :: y) = f (x, reduce f e y)
```

(* esempio: somma (sbagliato, + è infisso...) *)

```

- reduce + 0 [1,2,3];
Error: ...

(* esempio: somma (corretto) *)
- reduce (op +) 0 [1,2,3];
val it = 6 : int

(* esempio: media di  $[x_1, \dots, x_n] = x_1/n + \dots + x_n/n$  *)
- let
    fun f L (elem, accum) = elem / real(length L) + accum
    val lista = [1.0, 2.0, 3.0]
  in
    reduce (f lista) 0.0 lista
  end;
val it = 2.0 : real

```

Funzioni anonime → usando funzioni di ordine superiore sarebbe comodo passare funzioni semplici, senza nome e senza **let**

Le funzioni anonime si definiscono con **fn**

Sintassi: $\underbrace{\langle \text{expr} \rangle}_{\text{fn qui è una dichiarazione}} ::= \text{fn} \langle \text{argomento} \rangle \Rightarrow \langle \text{expr} \rangle$

```

- fn x => x+1;
val it = fn : int -> int

(* per sommare uno a tutti gli elementi di una lista *)
- map (fn x => x+1) [1,2,3];
val it = [2,3,4] : int list

- filter (fn x => (x mod 2) = 0) [10,11,12,13];
val it = [10,12] : int list

- fn x => fn y => x+y;
val it = fn : int -> int -> int

```

Curriente, vista l'assenza del nome, non possono essere ricorsive

Val vs fun $\rightarrow$ val è più generale di fun.

Le seguenti definizioni sono equivalenti

```
- fun f x = x + 1;  
val f = fn : int -> int  
  
(* equivalente al precedente *)  
- val f = fn x => x+1;  
val f = fn : int -> int
```

fun è **Zucchero sintattico**
(utile abbreviazione rispetto a val)

Ulteriori dettagli su currying

Riprendendo $\text{fun } f' \ x \ y = x + y$

$\downarrow$

scrittura equivalente a

$\downarrow$

$\text{fun } f' \ x = \text{fn } y \Rightarrow x + y$

$\text{val } f' = \text{fn} : \text{int} \rightarrow \text{int} \rightarrow \text{int}$

È anche possibile definire funzioni che effettuano il currying e la sua trasformazione inversa per una data funzione del tipo giusto

```
(* f deve accettare una coppia (x,y) *)  
val curry_2args = fn f => fn x => fn y => f (x, y)  
  
(* f deve essere del tipo f x y *)  
val uncurry_2args = fn f => fn (x, y) => f x y
```

Esempi di utilizzo

```
fun f (x,y) = x+y;  
  
val f' = curry_2args f; (* equivalente a f' x y = x+y *)
```

oppure

```
fun f' x y = x+y;  
  
val f = uncurry_2args f'; (* equivalente a f(x,y) = x+y *)
```


Tipo parametrico $\rightarrow$ equivalente ai Template di C++ o ai generics di Java

Già abbiamo visto un suo impiego con le liste, il costruttore $::$ che accetta ogni tipo.

Passando oltre:

- length;
val it = fn : 'a list $\rightarrow$ int

length prende una lista con tipo
'a non specificato
(e non interessa neanche il tipo, visto
che vogliamo la lunghezza)

Definizione di tipi parametrici:

```
- datatype 'a lista = vuota | nodo of ('a * 'a lista);  
datatype 'a lista = nodo of 'a * 'a lista | vuota
```

```
- datatype 'a bt = emptybt | btnode of ('a * 'a bt * 'a bt);  
datatype 'a bt = btnode of 'a * 'a bt * 'a bt | emptybt
```

La Type inference capirà il tipo

Per dire che il tipo parametrico possa supportare l'uguaglianza, si aggiunge
un '=' in più $\rightarrow$ 'a

■ Ogni tanto la type inference se ne accorge da sola

```
- fun diag (x,y) = x=y;  
val diag = fn : "a * "a -> bool  
  
- diag (1.0, 1.0);  
Error: operator and operand don't agree [equality type required]  
operator domain: ''Z * ''Z  
operand:         real * real
```

(ricordarsi che i reali non supportano l'uguaglianza...)

Encapsulation e interface

Le *signature* di ML sono equivalenti alle interfacce Java

Esempio: STACK

```
signature STACK =  
sig  
  type 'a stack // stack parameters  
  val empty : 'a stack  
  val push : ('a * 'a stack) -> 'a stack  
  val pop : 'a stack -> ('a * 'a stack)  
end;
```

⋮ permette l'assegnamento di un tipo

Ovviamente nulla è implementato o definito

Le *structure* definiscono tipi di dato astratti

Esempio di implementazione di STACK mediante una lista

```
structure Stack :> STACK =  
struct  
  type 'a stack = 'a list;  
  val empty = [];  
  fun push (x,s) = x :: s;  
  fun pop (x::s) = (x,s);  
end;
```

→ Stack deve implementare completamente STACK

I tipi di dato in Stack possono essere usati solo con le funzioni definite in STACK.

Ogni funzione definita nella structure non è visibile all'esterno (Encapsulation)

Anche se Stack nel suo cuore si sente una list, non è possibile trattarlo come tale...

```
- length [];  
val it = 0 : int  
  
- length Stack.empty;  
stdIn:39.1-39.19 Error: operator and operand don't agree  
operator domain: 'Z list  
operand:           'Y Stack.stack
```

... perché la structure non definisce nessuna funzione length e neanche come è fatto il tipo stack

Eccezioni e type checking

Anche ML ha le sue eccezioni e la gestione è integrata col type checking

- 3 div 0;
uncaught exception Div [divide by zero]

```
- fun pop(x::s) = (x,s);  
Warning: match nonexhaustive
```

```
x :: s => ...  
- pop [];  
uncaught exception Match [nonexhaustive match failure]
```

Funzionamento di questo esempio

- 1 - La type inference capisce che l'input di pop è una lista
- 2 - Il datatype lista ha i seguenti costruttori :: e []
- 3 - Pop ha solo un caso per :: capace di emettere il warning
- 4 - Il compilatore genera una eccezione Match nei casi mancanti
(In parole povere, non viene coperto il caso di lista vuota, e ML richiede che tutti i casi siano considerati)

Eccezioni personalizzate

```
exception EmptyStack;      (* dichiara una nuova eccezione *)  
  
fun pop(x::s) = (x,s)  
  | pop [] = raise EmptyStack;  (* come il throw di Java *)
```

Più chiaro rispetto al Match

Le eccezioni possono essere catturate e gestite con handle:

```
pop x  
handle EmptyStack =>  
  ( print "messaggio di errore specializzato";  
    raise EmptyStack );
```

Costrutto **handle** -> può essere collocato dopo un espressione che può generare un'eccezione

(3dur x) **handle** ...

Un solo **handle** può gestire diverse eccezioni:

```
<expression> handle
  <exception 1> => ...
| <exception 2> => ...
| ...
```

In ML ogni ordine è valido perché le eccezioni non rispettano una gerarchia e hanno un valore **distinto e unico**

Eccezioni con parametri -> aggiungono dettagli sull'errore

Esempio di eccezione con parametri:

```
exception SyntaxError of string
```

Questa eccezione può essere lanciata in diversi modi...

```
raise SyntaxError "Identifier expected"
```

```
raise SyntaxError "Integer expected"
```

... e il parametro "letto" col pattern matching

```
... handle SyntaxError x => ... (* qui si può usare x *)
```

Stiamo specificando il tipo di dato che verrà fornito al lancio dell'eccezione

x avrà il valore della stringa fornita

